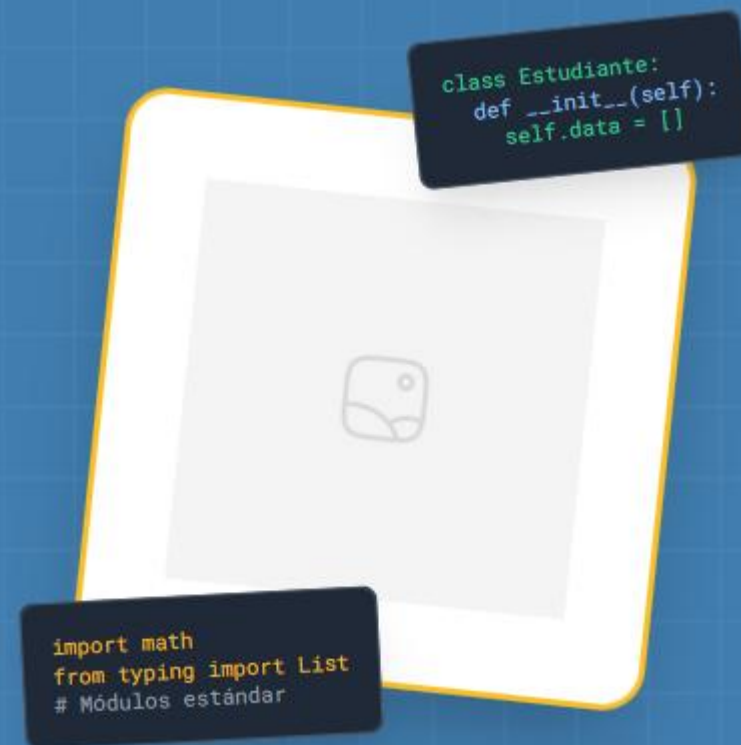


Tipos de Datos, Módulos y Bibliotecas

Domina la estructuración de código en Python



Parte 01

Tipos de Datos

Fundamentos esenciales para estructurar información en Python.



Objetivos de la sección

- Comprender tipos básicos y compuestos
- Diferenciar mutables vs inmutables
- Crear tipos personalizados con clases

Actividad Práctica

Diseña tu propia clase

Modelado de objetos del mundo real

Fundamentos

¿Qué son los Tipos de Datos?

La base de cualquier programa:
cómo representamos la
realidad en código.

Definición y Propósito

Determinan cómo se **almacena** la información en memoria y qué **operaciones** son válidas (ej. sumar números vs. concatenar textos).

Tipado Dinámico y Fuerte

Dinámico Variables infieren tipo **Fuerte** Restricción de operaciones

No necesitas declarar tipos (`int X`), pero Python no permite mezclar tipos incompatibles mágicamente.

Mutabilidad vs. Inmutabilidad

Mutables: Se modifican in-situ (Listas, Diccionarios).

Inmutables: No cambian tras crearse (Tuplas, Strings, Enteros). Impacta rendimiento y seguridad.

Tipos Incorporados vs. Personalizados

Python incluye tipos "baterías incluidas" (built-in) potentes, pero permite crear nuevos tipos mediante **Clases** para modelar problemas complejos.

Tipos de Datos en Python

Comparativa: Básicos vs. Compuestos

Tipos Básicos Atómicos

```
basicos.py

1 # --- Números (Inmutables) ---
2 edad = 42 # int
3 pi = 3.14159 # float
4
5 # --- Texto (Immutable) ---
6 saludo = "Hola Mundo"
7 # saludo[0] = "h" <- Error! No se puede cambiar
8
9 # --- Booleanos ---
10 es_valido = True
11 tiene_bugs = False
```

Nota:
Los tipos básicos son inmutables. Al "modificarlos", en realidad creas un nuevo objeto en memoria.

Tipos Compuestos Colecciones

```
compuestos.py

1 # --- Listas (Mutables) ---
2 nums = [1, 2, 3]
3 nums.append(4) # Modifica in-situ
4
5 # --- Tuplas (Inmutables) ---
6 punto = (10.5, 20.1)
7
8 # --- Dicionarios (Mutables) ---
9 usuario = {"nombre": "Ana", "id": 99}
10 usuario["rol"] = "Admin"
11
12 # --- Conjuntos (Mutables, Unicos) ---
13 tags = {"python", "dev", "python"}
14 print(tags) # {'python', 'dev'}
```

Crear Tipos Personalizados

Clases, Objetos y Atributos

```
1 class Persona:
2     #clase que representa una persona
3
4     #constructor con Type Hints
5     def __init__(self, nombre:str, edad:int):
6         self.nombre=nombre
7         self.edad=edad
8
9     #metodo de instancia
10    def saludar(self)->str:
11        return f"Hola, soy{self.nombre}"
12
13
14 #---USO (instanciacion)
15 p=Persona("Ana")
16 print(p.saludar())
17
```

Componentes Clave

`__init__` El Constructor

Método especial que se ejecuta automáticamente al crear una nueva instancia. Inicializa el estado del objeto.

`self` La Instancia Actual

Referencia explícita al objeto que se está manipulando. Permite acceder a sus atributos y métodos internos.

Atributos y Métodos

Los **atributos** (variables) almacenan datos y los **métodos** (funciones) definen el comportamiento.



Sugerencia: Dataclasses

Para clases que solo almacenan datos, usa el decorador `@dataclass`. Genera automáticamente `__init__`, `__repr__` y más.

```
from dataclasses import dataclass
@dataclass
class Punto:
    x: int
    y: int
```

Ejemplo Práctico: Clase Estudiante

Implementación completa de atributos y métodos

Definición de la Clase

Estructura

```
1 class Estudiante:
2     def __init__(self, nombre:str, notas:lista[float]):
3         self.nombre=nombre
4         self.notas=notas
5
6     def promedio(self)->float:
7         #calcula el promedio de las notas
8         if not self.notas:
9             return 0.0
10        return sum(self.notas)/len(self.notas)
11
12    def __str__(self)-> str:
13        #Representacion legible del objeto
14        prom=self.promedio()
15        return f"Estudiante({self.nombre}, promedio={prom:.1f})"
16
```

💡 **Tip:** El método especial `__str__` define cómo se ve el objeto cuando usamos `print()`.

Instanciación y Uso

Ejecución

```
main.py
1 # Crear un objeto (instancia)
2 alu = Estudiante("Lucía", [8, 9, 10])
3
4 # Acceder a atributos y métodos
5 print(alu.nombre)
6 print(f"Notas: {alu.notas}")
7
8 # __str__ se invoca automáticamente
9 print(alu)
```

TERMINAL OUTPUT

```
→ Lucía
→ Notas: [8, 9, 10]
→ Estudiante(Lucía, promedio=9.0)
```

Crea tu propia clase

Objetivo Principal

Definir una clase Producto

- ✓ **Define atributos** con anotaciones de tipo (ej. nombre: str, precio: float).
- ✓ Implementa el constructor `__init__` inicializando todos los atributos.
- ✓ Añade **validación básica**: asegura que el precio no sea negativo (lanza `ValueError` o ajusta a 0).
- ✓ Crea al menos **2 métodos útiles** (ej. `calcular_impuesto()`, `aplicar_descuento()`).
- ✓ Implementa `__str__` para mostrar una representación legible del objeto.



TIEMPO ESTIMADO

10 min



FORMATO

Parejas

Pregunta de comprensión

`list` es mutable y `tuple` es inmutable.

Falso

Verdadero 

Pregunta de comprensión

¿Qué método especial se ejecuta automáticamente al instanciar una clase?

a) `__new__`

b) `__init__`



c) `__repr__`

¿Para qué sirve el método `__str__` en una clase?

Devuelve una representación en **cadena de texto (string)** legible para humanos del objeto. Es lo que se muestra al usar `print(objeto)`.



Parte 02

Módulos

Estrategias para organizar, reutilizar y estructurar código en proyectos escalables.



Objetivos de la sección

- Entender qué es un módulo y un paquete
- Crear e importar módulos propios
- Conocer módulos estándar útiles

Actividad Práctica

Crea tu primer módulo

Implementación de una calculadora modular

Arquitectura

¿Qué es un Módulo?

La unidad fundamental de organización en Python: divide tu programa en partes manejables.

```
import math
print(math.pi)
# -> 3.14159...
```

Archivo Fuente (.py)

En su forma más simple, un módulo es cualquier archivo con extensión **.py** que contiene definiciones (funciones, clases) y sentencias de Python.

Organización Lógica

Orden **Estructura**

Permite agrupar código relacionado lógicamente. Por ejemplo, todas las funciones matemáticas en `math.py` o de conexión en `network.py`.

Reutilización (DRY)

Escribe el código una vez e impórtalo en múltiples scripts. Facilita el mantenimiento y evita la duplicación de lógica (*Don't Repeat Yourself*).

Espacios de Nombres (Namespaces)

Cada módulo define su propio ámbito. Las variables de `modulo_A` no colisionan con las de `modulo_B`, haciendo el código más seguro.

Crear e importar un módulo

Organización y reutilización de código

Definición del Módulo

ESTRUCTURA DEL PROYECTO

```
📁 proyecto/  
├── 📄 utilidades.py  
└── 📄 app.py
```

```
1 import math  
2 def area_circulo(radio):  
3     return math.pi * (radio**2)
```

Uso del Módulo

```
1 # Opción 1: Importar todo con alias  
2 import utilidades as u  
3  
4 r = 3  
5 print(f"Area: {u.area_circulo(r)}")  
6 # Salida: Area: 28.27...  
7  
8 # Opción 2: Importar función específica  
9 from utilidades import area_circulo  
10  
11 print(area_circulo(5))  
12 # Salida: 78.53...
```

Usa import ... as para evitar colisiones de nombres

Evita from modulo import *

Módulos Estándar Útiles

Biblioteca estándar de Python ("Batteries Included")

Cálculo y Estadística

Numérico

```
calculos.py

1 # --- Matemáticas (math) ---
2 import math
3
4 raiz = math.sqrt(16)
5 print(raiz) # Salida: 4.0
6 print(math.pi) # Salida: 3.1415...
7
8 # --- Estadística (statistics) ---
9 import statistics as stats
10
11 datos = [10, 20, 30, 20]
12
13 media = stats.mean(datos)
14 print(media) # Salida: 20
15 moda = stats.mode(datos)
16 print(modas) # Salida: 20
```

Sistema y Datos

Utilidades

```
utils.py

1 # --- Fechas (datetime) ---
2 from datetime import date
3 hoy = date.today() # 2023-10-27
4
5 # --- Archivos (pathlib) ---
6 from pathlib import Path
7
8 ruta = Path(".").resolve()
9 print(ruta) # /home/usuario/proyecto
10
11 # --- Datos (json) ---
12 import json
13
14 json_str = '{"id": 1, "activo": true}'
15 data = json.loads(json_str)
16 print(data["activo"]) # True
```

MATH

STATISTICS

DATETIME

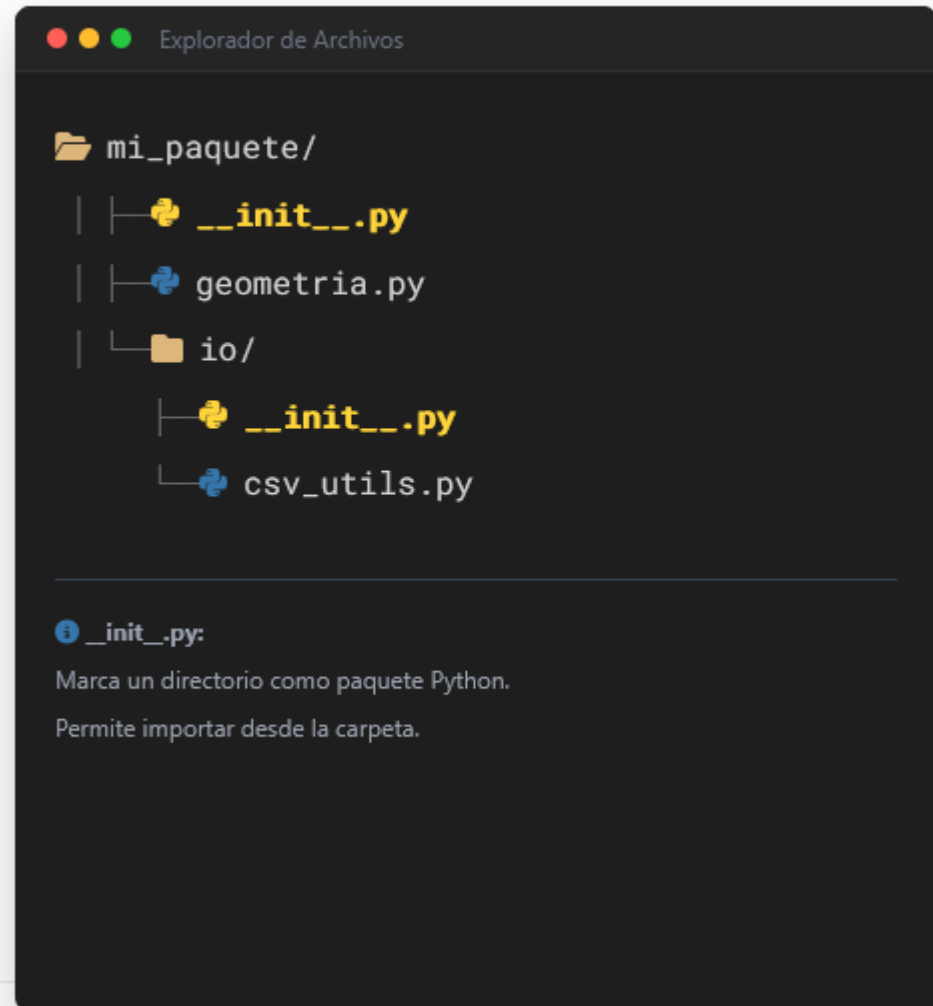
PATHLIB

JSON

Paquetes y `__init__.py`

Organización jerárquica y control de exportaciones

📁 Estructura del Paquete



Explorador de Archivos

```
mi_paquete/
├── __init__.py
├── geometria.py
└── io/
    ├── __init__.py
    └── csv_utils.py
```

__init__.py:
Marca un directorio como paquete Python.
Permite importar desde la carpeta.

📄 Código: `__init__.py`

```
mi_paquete/__init__.py

1 # Define qué se exporta con "from mi_paquete import *"
2 __all__ = ["geometria"]
3
4 # Código de inicialización opcional
5 print("Paquete 'mi_paquete' inicializado")
```

↔ Importes Relativos

```
.../io/csv_utils.py

1 # Importar desde el mismo paquete (punto .)
2 from . import lectores_base
3
4 # Importar desde el paquete padre (punto punto ..)
5 from ..geometria import Punto
6
7 # Nota: Solo funcionan dentro de paquetes
```

Actividad Práctica 2 Módulos y Paquetes

Tu primer módulo

Objetivo Principal

Crear archivo calculadora.py

- ✓ **Define las funciones básicas:** suma, resta, mult y div.
- ✓ **Manejo de errores:** En div, evita la división por cero (retorna None o mensaje).
- ✓ Crea un archivo nuevo main.py en la misma carpeta.
- ✓ **Importa tu módulo** usando import calculadora o from ... import
- ✓ Ejecuta y muestra en consola el resultado de al menos **3 operaciones** distintas.



TIEMPO ESTIMADO

12 min



FORMATO

Individual

Pregunta de comprensión

¿Qué estructura define técnicamente a un paquete en Python?

a) Un archivo .py muy grande

b) Una carpeta con `__init__.py` ★

c) Cualquier carpeta con código

Pregunta de comprensión

Diferencia clave al usar `from mod import func`:

a) Importa todo el módulo en RAM

b) Usas `func()` directo sin prefijo 

c) Es más lento que `import mod`

Pregunta de comprensión

¿Para qué sirve la lista `__all__` en `__init__.py`?:

a) Lista los módulos instalados

b) Define qué se importa con `*`



c) Bloquea el acceso al paquete

¿Cuándo conviene usar imports relativos (ej. `from . import lib`)?

Dentro de **paquetes organizados** para referirse a módulos "hermanos". Facilita mover o renombrar el paquete sin romper las referencias internas.



Parte 03

Bibliotecas Python

Potencia tus proyectos con el vasto ecosistema de paquetes de Python.



Objetivos de la sección

- Entender qué es un módulo y un paquete
- Crear e importar módulos propios
- Conocer módulos estándar útiles

Actividad Práctica

Explora una biblioteca
Consumo de API con request

¿Qué es un una Biblioteca?

Conjunto de código
listos para usar que
extienden las
capacidades de
Python.

```
import math
print(math.pi)
# -> 3.14159...
```

Definición y Propósito

Una biblioteca (o *library*) es una **colección empaquetada** de módulos y paquetes diseñada para resolver problemas comunes sin reinventar la rueda.

Distribución Pública

Se publican en repositorios centrales como **PyPI** (Python Package Index), permitiendo que cualquier desarrollador las instale y actualice fácilmente.

Ejemplos Populares

Requests HTTP fácil **numpy** Cálculo **pandas** Datos **flask** Web

Ventajas Clave

Aceleran el desarrollo, están probadas por la comunidad, suelen estar optimizadas y permiten enfocarse en la lógica específica de tu aplicación.

Instalación y Entornos

Gestión de paquetes con pip y aislamiento con venv

 **Entornos Virtuales** Aislamiento

 **Gestión con PIP** Instalación

terminal_setup.sh

```
1 # 1. Crear el entorno (se crea carpeta .venv)
2 python -m venv .venv
3
4 # 2. Activar entorno
5
6 # --- Windows (CMD) ---
7 .venv\Scripts\activate.bat
8
9 # --- Windows (PowerShell) ---
10 .venv\Scripts\Activate.ps1
11
12 # --- macOS / Linux ---
13 source .venv/bin/activate
14
15 # El prompt cambiará a (.venv)
```

terminal_install.sh

```
1 # 3. Actualizar pip (siempre buena idea)
2 python -m pip install --upgrade pip
3
4 # 4. Instalar librerías
5 pip install requests
6 pip install pandas==2.0.0
7
8 # 5. Verificar instalación
9 pip show requests
10
11 # 6. Prueba rápida (one-liner)
12 python -c "import requests; print(requests.__version__)"
```

Ecosistema

Bibliotecas Populares

Soluciones robustas para casi cualquier problema, listas para instalar desde PyPI.

```
> pip install requests +  
> pip install pandas + +  
> pip install fastapi + +
```

HTTP & Conectividad

Requests httpx

Clientes HTTP elegantes y sencillos para consumir APIs y servicios web.

Desarrollo Web / APIs

Flask fastapi django

Frameworks para construir desde microservicios rápidos hasta aplicaciones complejas.

Ciencia de Datos

Numpy pandas

Manipulación eficiente de arrays numéricos y análisis de estructuras de datos (DataFrames).

Visualización

Matplotlib seaborn

Creación de gráficos estadísticos, diagramas y visualizaciones de alta calidad.

Testing & Calidad

Pytest unittest

Frameworks para escribir pruebas unitarias y de integración escalables.

Validación de Datos

pydantic

Validación de datos y gestión de configuraciones aprovechando los type hints de Python.

Gestión de Dependencias

El archivo requirements.txt

> Comandos Clave

Terminal / Bash

```
1 # --- 1. Exportar dependencias ---
2 # Guarda las versiones instaladas en el archivo
3 pip freeze > requirements.txt
4
5 # --- Contenido generado (ejemplo) ---
6 requests==2.31.0
7 numpy==1.26.0
8 pandas==2.1.1
9
10 # --- 2. Instalar desde archivo ---
11 # Instala todo lo listado en el archivo
12 pip install -r requirements.txt
```

✓ Buenas Prácticas

Recomendaciones

📌 Versionado Estricto (Pinning)

Siempre especifica la versión exacta (==) para garantizar la reproducibilidad. Evita sorpresas cuando una biblioteca se actualice con cambios que rompan tu código.

```
flask==3.0.0 ✓ Correcto
flask X Evitar (instala la última)
```

✂ Herramientas Avanzadas

Para proyectos grandes, el archivo generado por `pip freeze` puede ser desordenado. Considera usar:

- **pip-tools:** Genera `requirements.txt` deterministas desde un archivo `requirements.in` más limpio.
- **Poetry / Pipenv:** Gestores modernos que manejan dependencias y entornos virtuales automáticamente.

Actividad Práctica 3 Módulos y Paquetes

Tu primer módulo

Objetivo Principal

Consumir una API pública usando requests

- ✓ Prepara el entorno: crea un **venv** e instala la biblioteca: `pip install requests`
- ✓ Realiza una petición **GET** a la API de GitHub:
<https://api.github.com/repos/python/cpython>
- ✓ Convierte la respuesta a diccionario usando `.json()`.
- ✓ Imprime los datos clave del repositorio:
Nombre, **Estrellas** (`stargazers_count`) y **Issues** (`open_issues`).



TIEMPO ESTIMADO

15 min



FORMATO

Individual

Ecosistema Python

Concepto	Definición & Propósito	Ejemplo Práctico
Clase / Tipo Core	Plantilla para crear objetos. Define atributos (datos) y métodos (comportamiento).	<code>class Estudiante: ...</code>
Módulo Core	Un único archivo <code>.py</code> que contiene código reutilizable (funciones, clases).	<code>import utilidades</code>
Paquete Core	Directorio que contiene módulos y un archivo especial <code>__init__.py</code> .	<code>from mi_paquete import modulo</code>
Biblioteca Ext	Colección de paquetes publicada para resolver problemas específicos (distribuida en PyPI).	<code>import requests</code>
Entorno Virtual Tool	Espacio aislado para gestionar dependencias de un proyecto sin conflictos.	<code>python -m venv .venv</code>
Pip Tool	El instalador de paquetes estándar para Python. Descarga desde PyPI.	<code>pip install pandas</code>

Tarea Final & Conclusiones

Desafío Final

1. **Clase CuentaBancaria:** Implementa métodos para depositar y retirar con validaciones (saldo insuficiente, monto negativo). Incluye método `__str__`.
2. **Paquete "reportes":** Crea una estructura de paquete con `__init__.py` y dos módulos: `csv_utils.py` y `json_utils.py`.
3. **Dependencias:** Genera el archivo `requirements.txt` de tu entorno virtual usando `pip freeze`.
4. **Publicación:** Sube tu código a un repositorio (GitHub/GitLab) y comparte el enlace en el foro del curso.